

task_scheduler support for parallel bulk execution

Document #:	P3927R0 [Latest] [Status]
Date:	2026-01-14
Project:	Programming Language C++
Audience:	SG1 Concurrency and Parallelism Working Group LEWG Library Evolution Working Group LWG Library Working Group
Reply-to:	Eric Niebler < eric.niebler@gmail.com >

Contents

- [1 Synopsis](#)
- [2 Background](#)
- [3 Implementation Experience](#)
- [4 Proposed Wording](#)

§ 1 Synopsis

By default, instances of the coroutine type `std::execution::task` store the “current” scheduler in type-erased scheduler wrapper called `std::execution::task_scheduler`. As with other type-erased wrappers, the goal of `std::execution::task_scheduler` is presumably to behave as much like a drop-in replacement for the object it wraps as is possible.

The `task_scheduler` falls short of this ideal in one respect: if a `task_scheduler` wraps a `parallel_scheduler` and is used to launch parallel work with a bulk sender, the work is not parallelized as it would be had a `parallel_scheduler` been used directly. That is because the `task_scheduler` does not treat the bulk algorithms specially, as `parallel_scheduler` does.

Fortunately, the `parallel_scheduler` has been specified in such a way that the `task_scheduler` can reuse its back-end helpers, making the job of specifying an improved `task_scheduler` much easier.

§ 2 Background

Like `task_scheduler`, the `parallel_scheduler` is a type-erased wrapper for a scheduler-like object. It uses the abstract base classes `parallel_scheduler_backend` and `receiver_proxy` to punch the `schedule`, `bulk_chunked`, and `bulk_unchunked` operations through the type-erased interface. These are precisely the operations we would like `task_scheduler` to handle.

Currently, `task_scheduler` is specified to have an exposition-only member `sch_` of type `shared_ptr<void>`. If this is changed to `shared_ptr<parallel_scheduler_backend>`, then the bulk algorithms can dispatch through `sch_>schedule_bulk_chunked(...)` and `sch_>schedule_bulk_unchunked(...)` and be accelerated for free.

Well ok, not exactly free; *some* work is needed:

- We need a class that inherits `parallel_scheduler_backend` and implements its abstract interface in terms of a concrete scheduler, like:

```
template<scheduler Sch>
struct task_scheduler_backend : parallel_scheduler_backend
{
    // exposition only
    explicit task_scheduler_backend(Sch sch) :
        sched_(std::move(sch)) {}

    void schedule(receiver_proxy& r, span<byte> s) noexcept
        override;
    void schedule_bulk_chunked(size_t shape, bulk_item_re-
        ceiver_proxy& r,
                                span<byte> s) noexcept over-
        ride;
    void schedule_bulk_unchunked(size_t shape, bulk_item_re-
        ceiver_proxy& r,
                                span<byte> s) noexcept over-
        ride;

    Sch sched_;
};
```

The `schedule` override would connect the result of calling `execution::schedule(sched_)` with a receiver that wraps the `receiver_proxy` and then calls `start` on the resulting operation state.

The `schedule_bulk_[un]chunked` overrides would construct a bulk sender whose predecessor is essentially the `just()` sender, but with a value completion scheduler of `sched_`. It would then connect that bulk sender with a receiver that wraps the `bulk_item_receiver_proxy` and calls `start` on the resulting operation state. Since the predecessor sender has `sched_` as its value completion scheduler, `connect` will use `sched_`'s domain to transform the bulk sender before connecting it with the receiver, causing the sender to use a custom implementation as appropriate.

- We also need `task_scheduler` to have a completion domain with a `transform_sender` member function that accepts vanilla `bulk_[un]chunked` senders and

transforms them so that they use `sch_>schedule_bulk_chunked(...)` and `sch_>schedule_bulk_unchunked(...)`.

```
struct task_scheduler_domain : default_domain {
    template<class BulkSndr, class Env>
        static constexpr auto transform_sender(set_value_t,
            BulkSndr&& bulk_sndr, const Env& env) noexcept;
};
```

This member function would be constrained to accept only bulk_[un]chunked senders and would return a new sender that, when connected and started, would connect and start bulk_sndr's predecessor sender. Error and stopped completions are forwarded to the receiver. Value completions are used to construct a bulk_item_receiver_proxy which is passed to `sch_>schedule_bulk_chunked(...)`.

§ 3 Implementation Experience

The proposed solution has been implemented in NVIDIA's [CCCL](https://github.com/NVIDIA/cccl) library. The relevant pull request can be found at <https://github.com/NVIDIA/cccl/pull/5975>, and the source for the task_scheduler is [here](#).

§ 4 Proposed Wording

[Editor's note: Change 33.13.5 [exec.task.scheduler] as follows:]

```
namespace std::execution {
    class task_scheduler {
        class ts_sender ts_domain;                // exposition only

        template<receiver R>
        class state; // exposition only

        template<scheduler Sch>
        class backend_for; // exposition only
    public:
        using scheduler_concept = scheduler_t;

        template<class Sch, class Allocator = allocator<void>>
            requires (!same_as<task_scheduler, remove_cvref_t<Sch>>)
                && scheduler<Sch>
        explicit task_scheduler(Sch&& sch, Allocator alloc = {});

        ts_sendersee below schedule();

        friend bool operator==(const task_scheduler& lhs, const
            task_scheduler& rhs) noexcept;

        template<class Sch>
            requires (!same_as<task_scheduler, Sch>) && scheduler<Sch>
        friend bool operator==(const task_scheduler& lhs, const
            Sch& rhs) noexcept;
```

```

private:
    shared_ptr<voidparallel_scheduler_backend> sch_; // exposition only
                                                    // see
                                                    [exec.sysctxrepl.psb].
};
}

```

1. `task_scheduler` is a class that models scheduler (33.6 [exec.sched]). Given an object `s` of type `task_scheduler`, let $SCHED(s)$ be the `sched` member of the object owned by `s.sch_`. The expression `get forward progress guarantee(s)` is equivalent to `get forward progress guarantee(SCHED(s))`. The expression `get completion domain<set value t>(s)` is equivalent to `task_scheduler::ts-domain()`.

```

template<class Sch, class Allocator = allocator<void>>
    requires(!same_as<task_scheduler, remove_cvref_t<Sch>>) &&
        scheduler<Sch>
explicit task_scheduler(Sch&& sch, Allocator alloc = {});

```

2. *Effects*: Initialize `sch_` with `allocate_shared<backend-for<remove_cvref_t<Sch>>>(alloc, std::forward<Sch>(sch))`.
3. *Recommended practice*: Implementations should avoid the use of dynamically allocated memory for small scheduler objects.
4. *Remarks*: Any allocations performed by ~~construction of `ts-sender` or `state-objects` resulting from~~ calls on `*this` are performed using a copy of `alloc`.

```

ts-sender-schedule();

```

5. ~~*Effects*: Returns an object of type `ts-sender` containing a sender initialized with `schedule(SCHED(*this))`.~~

```

bool operator==(const task_scheduler& lhs, const task_scheduler& rhs) noexcept;

```

6. *Effects*: Equivalent to: `return lhs == SCHED(rhs);`

```

template<class Sch>
    requires (!same_as<task_scheduler, Sch>) && scheduler<Sch>
bool operator==(const task_scheduler& lhs, const Sch& rhs) noexcept;

```

7. *Returns*: `false` if the type of $SCHED(lhs)$ is not `Sch`, otherwise $SCHED(lhs) == rhs$.

[Editor's note: Remove paragraphs 8-12 and add the following paragraphs:]

8. For an lvalue *r* of type derived from `receiver_proxy`, let *WRAP-RCVR(r)* be an object of a type that models receiver and whose completion handlers result in invoking the corresponding completion handlers of *r*.

```
namespace std::execution {
    template<scheduler Sch>
        class task_scheduler::backend_for : public parallel_scheduler_backend {
            // exposition only
        public:
            explicit backend_for(Sch sch) : sched_(std::move(sch)) {}

            void schedule(receiver_proxy& r, span<byte> s) noexcept override;
            void schedule_bulk_chunked(size_t shape, bulk_item_receiver_proxy& r,
                                         span<byte> s) noexcept override;
            void schedule_bulk_unchunked(size_t shape, bulk_item_receiver_proxy& r,
                                         span<byte> s) noexcept override;

            Sch sched_; // exposition only
        };
}
```

9. Let *just-sndr-like* be a sender whose only value completion signature is `set_value_t()` and for which the expression `get_completion_scheduler<set_value_t>(get_env(just-sndr-like)) == sched_` is true.

```
void schedule(receiver_proxy& r, span<byte> s) noexcept override;
```

10. *Effects:* Constructs an operation state *os* with `connect(schedule(sched_), WRAP-RCVR(r))` and calls `start(os)`.

```
void schedule_bulk_chunked(size_t shape, bulk_item_receiver_proxy& r,
                           span<byte> s) noexcept override;
```

11. *Effects:* Let *chunk_size* be an integer less than or equal to *shape*, let *num_chunks* be $(\text{shape} + \text{chunk_size} - 1) / \text{chunk_size}$, and let *fn* be a function object such that for an integer *i*, *fn*(*i*) calls `r.execute(i * chunk_size, m)`, where *m* is the lesser of $(i + 1) * \text{chunk_size}$ and *shape*. Constructs an operation state *os* as if with `connect(bulk(just-sndr-like, par, num_chunks, fn), WRAP-RCVR(r))` and calls `start(os)`.

```
void schedule_bulk_unchunked(size_t shape, bulk_item_receiver_proxy& r,
                             span<byte> s) noexcept override;
```

12. *Effects*: Let *fn* be a function object such that for an integer *i*, *fn*(*i*) is equivalent to *r.execute*(*i*, *i* + 1). Constructs an operation state *os* as if with *connect*(*bulk*(*just-sndr-like*, *par*, *shape*, *fn*), *WRAP-RCVR*(*r*)) and calls *start*(*os*).

see *below* *schedule*();

13. *Returns*: a prvalue *ts-sndr* whose type models sender such that:

- (8.1) — *get_completion_scheduler*<*set_value_t*>(*get_env*(*ts-sndr*)) is equal to **this*.
- (8.2) — *get_completion_domain*<*set_value_t*>(*get_env*(*ts-sndr*)) is expression-equivalent to *ts-domain*()).
- (8.3) — If a receiver *rcvr* is connected to *ts-sndr* and the resulting operation state is started, calls *sch_*→*schedule*(*r*, *s*), where
 - (8.3.1) — *r* is a proxy for *rcvr* with base *system_context_replaceability::receiver_proxy* (33.15 [exec.par.scheduler]) and
 - (8.3.2) — *s* is a preallocated backend storage for *r*.
- (8.4) — *completion_signatures_of_t*<*Sndr*> denotes:

```
completion_signatures<
    set_value_t(),
    set_error_t(error_code),
    set_error_t(exception_ptr),
    set_stopped_t()
>
```

```
namespace std::execution {
    class task_scheduler::ts-domain : public default_domain {
    public:
        template<class BulkSndr, class Env>
            static constexpr auto transform_sender(set_value_t,
                BulkSndr&& bulk_sndr, const Env& env)
                noexcept;
    };
}
```

```
template<class BulkSndr, class Env> // exposition only
    static constexpr see_below transform_sender(BulkSndr&& bulk_s-
        ndr, const Env& env)
        noexcept;
```

14. *Constraints*: *sender_in*<*BulkSndr*, *Env*> is true, *auto*(*std::forward*<*BulkSndr*>(bulk_sndr)) is well-formed, and either *sender-for*<*BulkSndr*, *bulk_chunked_t*> or *sender-for*<*BulkSndr*, *bulk_unchunked_t*> is true.

15. *Effects*: Equivalent to:

```

auto& [_, data, child] = bulk_sndr;
auto& [_, shape, fn] = data;
auto sch = call-with-default(get_completion_scheduler<set_-
    value_t>,
                                not-a-scheduler(),
                                get_env(child), FWD-ENV(env));
return e;

```

where e is *not-a-sender()* if the type of sch is not `task_scheduler`; otherwise, it is a prvalue whose type models sender such that, if it is connected to $rcvr$ and the resulting operation state is started, $child$ is connected to an unspecified receiver R and started. If $child$ completes with an error or a stopped completion, the completion operation is forwarded unchanged to $rcvr$. Otherwise, let $args$ be a pack of lvalue subexpressions designating objects decay-copied from the value result datums. Then

- (15.1) — If $bulk_sndr$ was the result of the evaluation of an expression equivalent to `bulk_chunked(child, policy, shape, f)` or a copy of such, then $sch_ \rightarrow schedule_bulk_chunked(shape, r, s)$ is called where r is a bulk chunked proxy (33.15 [exec.-par.scheduler]) for $rcvr$ with callable f and arguments $args$, and s is a preallocated backend storage for r .
- (15.2) — Otherwise, calls $sch_ \rightarrow schedule_bulk_unchunked(shape, r, s)$ where r is a bulk unchunked proxy for $rcvr$ with callable f and arguments $args$, and s is a preallocated backend storage for r .

16. *Recommended practice:* The returned sender should hold references to the parts of `bulk_sndr` that it needs.

17. *Remarks:* The expression `get_env(R)` is expression-equivalent to `FWD-ENV(get_env(rcvr-copy))`, where *rcvr-copy* is an lvalue sub-expression designating an object decay-copied from $rcvr$.